

NSAPDEV Server Application Project

Software Architecture Design Document

0. Cover Page

Course: NSAPDEV (Server Application Development)

Term/AY: 2nd Term AY 2025-2026

Project Title: Concurrent Syslog Analytics Server ("Mini-Splunk")

Team Members: Joshua Benedict B. Co, Reyvin Matthew T. Tan

1. System Overview

- **Purpose:** Mini-Splunk is a lightweight, centralized log management server that ingests, parses, and indexes standard syslog files from multiple concurrent CLI clients over a TCP network. It stores parsed log entries in a shared in-memory data structure and allows clients to query that data in real time. The system is designed as a simplified analogue of enterprise SIEM (Security Information and Event Management) tools such as Splunk, demonstrating core server application development concepts including socket programming, multithreading, and mutual exclusion [3].
- **Core Capabilities:**
 - File Ingestion: Accepts raw syslog file content streamed from one or more CLI clients simultaneously via TCP using the INGEST command, which sends `UPLOAD|<filesize>|<file_content_stream>` to the server.
 - RFC 3164 Parsing: Automatically extracts structured fields (timestamp, hostname, daemon, severity, message) from each log line using `SYSLOG_REGEX`, a compiled regular expression conforming to the BSD Syslog Protocol (RFC 3164) [1].
 - Concurrent Querying: Supports multiple simultaneous read queries via `SEARCH_DATE`, `SEARCH_HOST`, `SEARCH_DAEMON`, `SEARCH_SEVERITY`, and `SEARCH_KEYWORD / COUNT_KEYWORD`, while safely handling concurrent write operations (`INGEST`, `PURGE`) through `threading.RLock()` [2].
 - Administrative Control: Provides a `PURGE` command that sends `ADMIN|PURGE` to the server, which exclusively locks and clears the entire `log_store` in-memory list.

2. High-Level Architecture Model

- **Architectural Style:** The system follows a Client-Server model with a multithreaded backend [4]. The server (`server.py`) acts as the centralized Indexer: it maintains all shared state in `log_store` and executes all data operations. The client (`client.py`) acts as a stateless Forwarder: it reads local syslog files from disk, translates user commands typed at the `mini-splunk>` prompt into pipe-delimited TCP protocol messages, and renders server responses to the terminal. All communication occurs over raw TCP sockets on port 9514.

- **System Components:**

- **The Forwarder (CLI Client):** Responsible for parsing user commands from the interactive mini-splunk> prompt, reading local syslog files from disk, opening a new TCP connection per command, serializing the command into the pipe-delimited protocol (e.g. QUERY|SEARCH_HOST|SYSSVR1), and printing the decoded server response to the terminal.
- **The Indexer (Server):** Responsible for binding to port 9514 via socket.bind(), accepting TCP connections on the main thread via socket.accept(), dispatching each connection to a dedicated worker thread via threading.Thread(target=handle_client), safely reading from and writing to log_store under store_lock (a threading.RLock()), and executing all search and administrative functions.

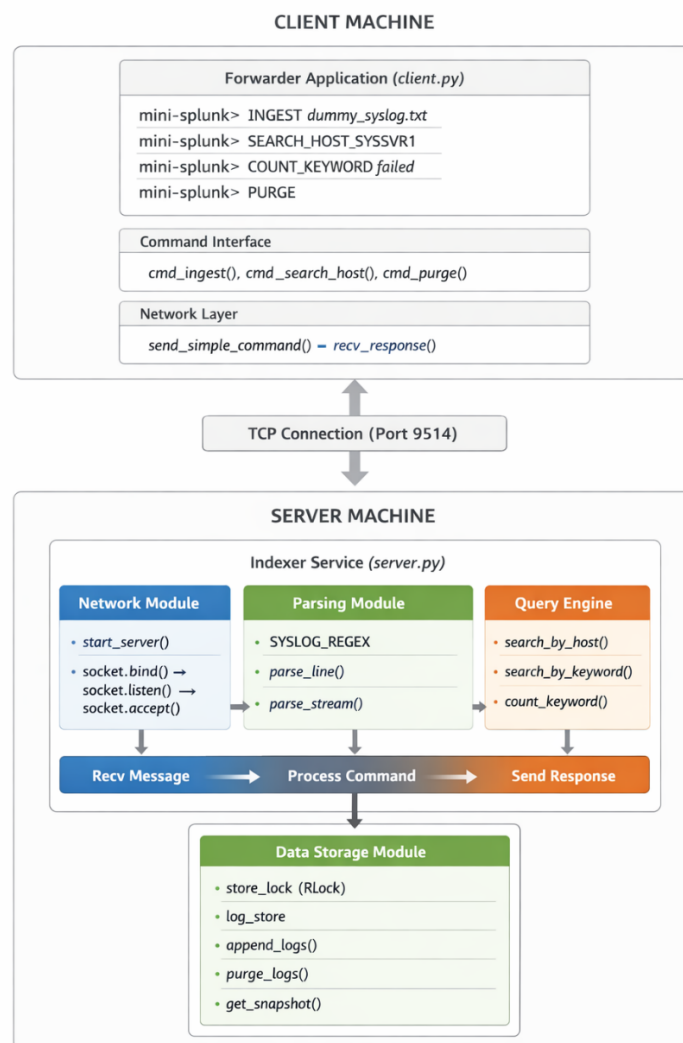


Figure 1: Mini-Splunk Distributed Architecture (Client-Server Design)

Overall system architecture of the Mini-Splunk distributed log analysis platform, illustrating the client forwarder and server indexer modules communicating through a TCP connection.

3. Concurrency and Threading Model

Connection Handling: The server uses a Thread-per-Connection model [4]. The main listener thread runs an infinite `socket.accept()` loop inside `start_server()`. Each time a client connects, the main thread immediately passes the accepted socket object to a newly spawned `threading.Thread(target=handle_client, args=(conn, addr), daemon=True)` and loops back to `socket.accept()`. It is never blocked waiting for a client to finish. Each worker thread is responsible for calling `recv_message(conn)` to read one command, calling `process_command(message)` to execute it, calling `send_response(conn, result)` to return the result, and then closing the socket. Worker threads are set with `daemon=True` so they are automatically reclaimed when the server process exits.

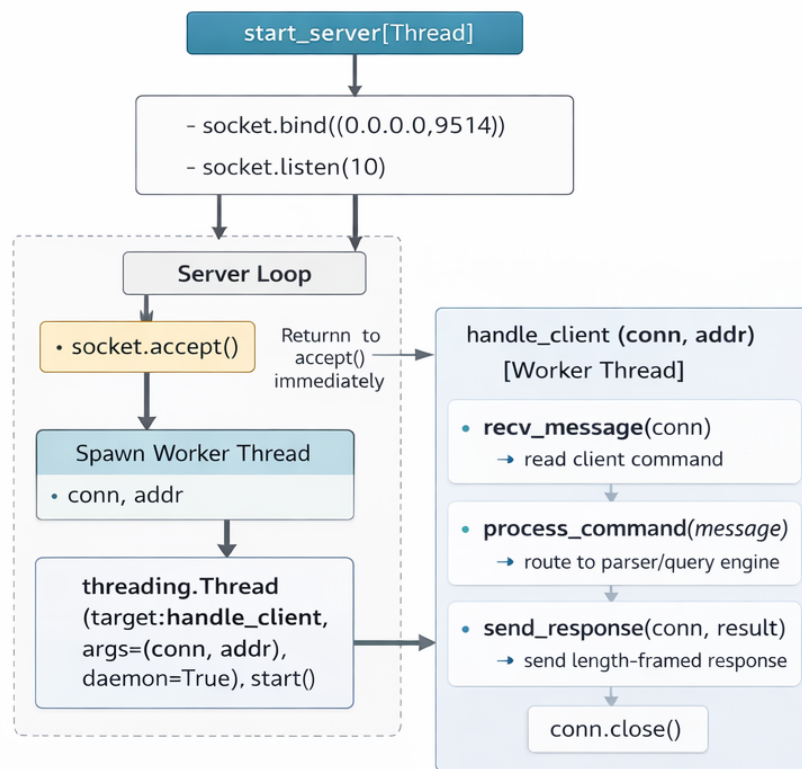


Figure 2: Multi-Threaded Server Connection Handling

Multi-threaded server design showing how the main server thread continuously accepts incoming connections and spawns worker threads to handle client requests concurrently.

- **Isolation & Synchronization:** The global `log_store` list is a Critical Section [5] because multiple worker threads may attempt to read, write (INGEST), or clear (PURGE) it concurrently. Without synchronization, simultaneous writes could corrupt list state, and a read during a write could return a partially updated or inconsistent snapshot.
 - `store_lock = threading.RLock()` (Reentrant Lock) ensures mutual exclusion during all write operations (`append_logs()`, `purge_logs()`) and during the snapshot copy inside `get_snapshot()`. A Reentrant Lock was chosen over a plain `threading.Lock()` because it allows the same thread to acquire the lock multiple times without deadlocking [2].
 - All query functions call `get_snapshot()`, which holds the lock only long enough to copy the list. The filter runs on the local copy with no lock held, minimising contention and allowing reader threads to proceed in parallel after the snapshot is taken.

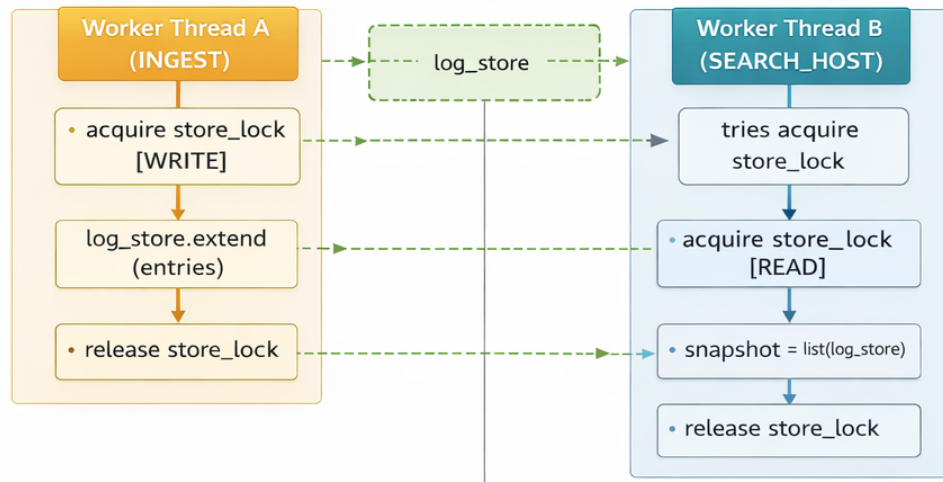


Figure 3: Concurrency Control Using Re-entrant Locks

Synchronization mechanism used in the server's shared log storage. Write operations obtain exclusive access to the log store, while read operations create snapshots to minimize lock contention.

4. Domain and Functional Decomposition

The server is decomposed into four logically isolated modules, each with a single clear responsibility. This mirrors the Separation of Concerns principle and makes the system easier to test and extend independently.

- **Network Module:** Handles `socket.bind()`, `socket.listen()`, and `socket.accept()`. `recv_message()` uses two-phase reading: it first reads a newline-terminated header to identify the command type; if the header begins with `UPLOAD`, it then reads exactly `<filesize>` further bytes for the syslog file body. `send_response()` wraps every reply in `<byte_length>\n<body>` framing so multi-line query results are always received atomically by the client's `recv_response()`.

```

recv_message(conn):

    READ bytes until '\n' --> header_str

    IF header_str starts with "UPLOAD|":

        PARSE filesize (e.g. "UPLOAD|1596|" -> 1596)

        READ exactly filesize bytes --> content_buf

        RETURN "UPLOAD|{filesize}|{content}"

    ELSE:

        RETURN header_str (e.g. "QUERY|SEARCH_HOST|SYSSVR1")

send_response(conn, response):

    encoded = response.encode("utf-8")

    conn.sendall( f"{len(encoded)}\n".encode() )

    conn.sendall( encoded )

```

Table 1: MessageProtocol (Communication protocol)

Handles the wire protocol for receiving and sending messages between components

- Parsing Module:** Contains the regular expression (SYSLOG_REGEX) logic to convert raw syslog strings into structured dictionaries [6]. `parse_stream()` splits the received file content on newlines and calls `parse_line()` for each line, silently discarding lines that do not match. Severity is derived by masking the PRI field: `severity = PRI & 0x07`, then looked up in `SEVERITY_MAP` [1].

```

SYSLOG_REGEX pattern:

^<(\d+)>                Group 1: PRI    e.g. 134
(\w{3}\s+\d{1,2}\s+\d{2}:\d{2}:\d{2}) Group 2: TIMESTAMP e.g. Feb 22 00:05:38
\s+(\S+)                Group 3: HOSTNAME e.g. SYSSVR1
\s+(\S+)?(?:\[ \d+\])?:\s+ Group 4: DAEMON  e.g. systemd (PID stripped)
(.+)$                  Group 5: MESSAGE  e.g. Started OpenBSD...

parse_line(raw_line):

    match = SYSLOG_REGEX.match(raw_line)

    IF no match: RETURN None

```

```
severity = SEVERITY_MAP[ int(group(1)) & 0x07 ]
```

```
RETURN {
```

```
    "timestamp": group(2),
```

```
    "hostname": group(3),
```

```
    "daemon": group(4),
```

```
    "severity": severity,
```

```
    "message": group(5)
```

```
}
```

```
parse_stream(content):
```

```
    FOR each line in content.splitlines():
```

```
        entry = parse_line(line)
```

```
        IF entry is not None: append to entries
```

```
    RETURN entries
```

Table 2: SyslogParser (Parsing logic)

Contains regex patterns and parsing logic for RFC 3164 syslog messages

- **Data Storage Module:** `log_store`, `store_lock`, `append_logs()`, `purge_logs()`, `get_snapshot()`
 - Manages the shared memory state (a global Python List of Dictionaries, `log_store = []`) and encapsulates all locking mechanisms via `store_lock = threading.RLock()`. No other module accesses `log_store` directly — all reads and writes go through one of these three functions, keeping the locking discipline centralised in one place [2].

```
append_logs(entries):    -- called on INGEST
```

```
    WITH store_lock:
```

```
        log_store.extend(entries)
```

```
purge_logs():           -- called on ADMIN|PURGE
```

```
    WITH store_lock:
```

```
        log_store.clear()
```

```

get_snapshot():          -- called by every QUERY function

    WITH store_lock:

        RETURN list(log_store) -- shallow copy; lock released immediately after

```

Table 3: LogStore (Storage management)

Manages the in-memory storage of parsed log entries with thread-safe operations

- **Query Engine Module:** search_by_date(), search_by_host(), search_by_daemon(), search_by_severity(), search_by_keyword(), count_keyword(), format_entries()
 - Houses the functions for filtering the data (SEARCH_DATE, SEARCH_HOST, SEARCH_DAEMON, SEARCH_SEVERITY, SEARCH_KEYWORD, COUNT_KEYWORD). All functions follow the same stateless pattern: obtain a thread-safe snapshot from the Data Storage Module, apply a list comprehension filter, and pass results to format_entries(). count_keyword() returns a plain integer string instead of formatted log lines.

```

search_by_host(hostname):

    snapshot = get_snapshot()

    results = [e for e in snapshot
                IF e["hostname"].lower() == hostname.lower()]

    RETURN format_entries(results) -- "NO_RESULTS" if empty


count_keyword(word):

    snapshot = get_snapshot()

    total = COUNT e in snapshot WHERE word.lower() in e["message"].lower()

    RETURN str(total)


format_entries(entries):

    FOR each entry:

        OUTPUT "[{timestamp}] {hostname} {daemon} [{severity}] {message}"

```

Table 4: LogQuery (Query operations)

Provides search and query functionality against the stored log entries

5. Shared Data State Layout

Define exactly what the parsed data looks like in the server's RAM.

- **Data Structure:** A single global Python List of Dictionaries (log_store) held in the server's process memory. There is no file-backed or database persistence, all data exists only for the lifetime of the server process. The list grows through append_logs() on every INGEST command and is fully cleared by purge_logs() on every PURGE command.
- **Schema:** Each element of log_store is a Python dictionary with exactly five string-valued keys [1]:

```
{
  "timestamp": "Feb 22 00:05:38",
  "hostname": "SYSSVR1",
  "daemon": "systemd",
  "severity": "INFO",
  "message": "Started OpenBSD Secure Shell server daemon"
}
```

Key	Derived From	Example Value
timestamp	RFC 3164 TIMESTAMP — regex group 2	"Feb 22 00:05:38"
hostname	RFC 3164 HOSTNAME — regex group 3	"SYSSVR1"
daemon	RFC 3164 TAG — regex group 4, PID stripped	"systemd"
severity	RFC 3164 PRI — PRI & 0x07 mapped via SEVERITY_MAP	"INFO"
message	RFC 3164 MSG — regex group 5	"Started OpenBSD..."

Table 5: LogEntrySchema (Data structure definition)

Defines the structure/fields of a parsed syslog entry

RFC 3164 encodes both facility and severity in the PRI field: $PRI = (Facility \times 8) + Severity$. Only the lower 3 bits determine severity [1]:

PRI & 0x07	SEVERITY_MAP Label
0	EMERG
1	ALERT
2	CRIT
3	ERR
4	WARNING
5	NOTICE
6	INFO
7	DEBUG

Table 6: SyslogSeverityMap (Severity mapping/lookup)

Maps priority values to severity labels as defined in RFC 3164

6. Client-Server Communication Protocol

All messages are exchanged over a raw TCP stream on port 9514. The protocol is text-based and pipe-delimited (|). The client opens a new TCP connection per command and closes it after receiving the response. Every server response uses length-prefix framing (<byte_length>\n<response_body>), so the client's `recv_response()` always reads the complete payload regardless of newlines in the body.

Client Trigger	Command	Data Sent to Server (Protocol)	Server Action / Expected Response
INGEST		UPLOAD <filesize> <file_content_stream>	Calls <code>parse_stream()</code> , acquires write lock via <code>append_logs()</code> , appends to <code>log_store</code> . Returns "SUCCESS: Ingested N log entries into the store"
SEARCH_DATE		QUERY SEARCH_DATE <date>	Acquires read lock via <code>get_snapshot()</code> , calls <code>search_by_date()</code> , filters by date, returns formatted string of logs.
SEARCH_HOST		QUERY SEARCH_HOST <hostname>	Acquires read lock via <code>get_snapshot()</code> , calls <code>search_by_host()</code> , filters by hostname, returns formatted string of logs.
SEARCH_DAEMON		QUERY SEARCH_DAEMON <daemon>	Acquires read lock via <code>get_snapshot()</code> , calls <code>search_by_daemon()</code> , filters by daemon, returns formatted string of logs.

SEARCH_SEVERITY	QUERY SEARCH_SEVERITY <level>	Acquires read lock via get_snapshot(), calls search_by_severity(), filters by severity, returns formatted string of logs.
SEARCH_KEYWORD	QUERY SEARCH_KEYWORD <word>	Acquires read lock via get_snapshot(), calls search_by_keyword(), filters message by word, returns logs.
COUNT_KEYWORD	QUERY COUNT_KEYWORD <word>	Acquires read lock via get_snapshot(), calls count_keyword(), counts occurrences, returns integer string.
PURGE	ADMIN PURGE	Acquires exclusive write lock via purge_logs(), clears log_store, returns success message.

Sequence Diagram — INGEST followed by SEARCH_HOST:

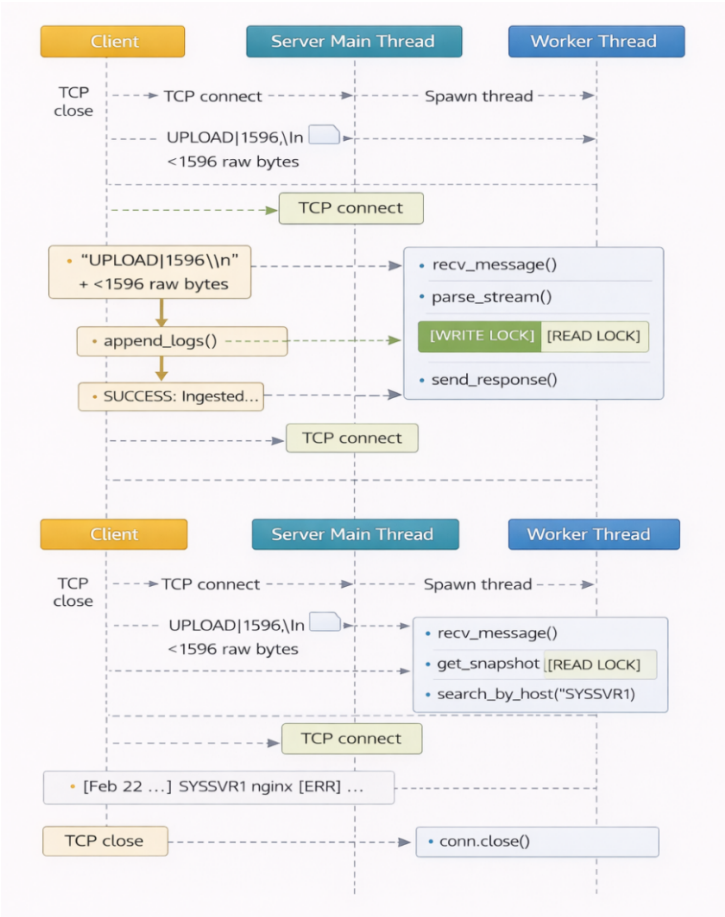


Figure 4: Client–Server Communication Sequence

Message sequence diagram illustrating the communication workflow between the Mini-Splunk client and server during log ingestion and query operations.

7. References

- [1] C. Lonvick, "The BSD Syslog Protocol," RFC 3164, Internet Engineering Task Force, Aug. 2001. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3164>
- [2] Python Software Foundation, "threading — Thread-based parallelism," Python 3 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/threading.html>
- [3] Python Software Foundation, "socket — Low-level networking interface," Python 3 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/socket.html>
- [4] B. Hall, "Beej's Guide to Network Programming: Using Internet Sockets," ver. 3.1.11, 2023. [Online]. Available: <https://beej.us/guide/bgnet/>
- [5] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, ver. 1.10. Arpaci-Dusseau Books, 2023. [Online]. Available: <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- [6] Python Software Foundation, "re — Regular expression operations," Python 3 Documentation, 2024. [Online]. Available: <https://docs.python.org/3/library/re.html>
- [7] R. Gerhards, "The Syslog Protocol," RFC 5424, Internet Engineering Task Force, Mar. 2009. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5424>